

Creator Communication Platform — System Design

Assignment v2 | Shashank Telkhade | April 2026

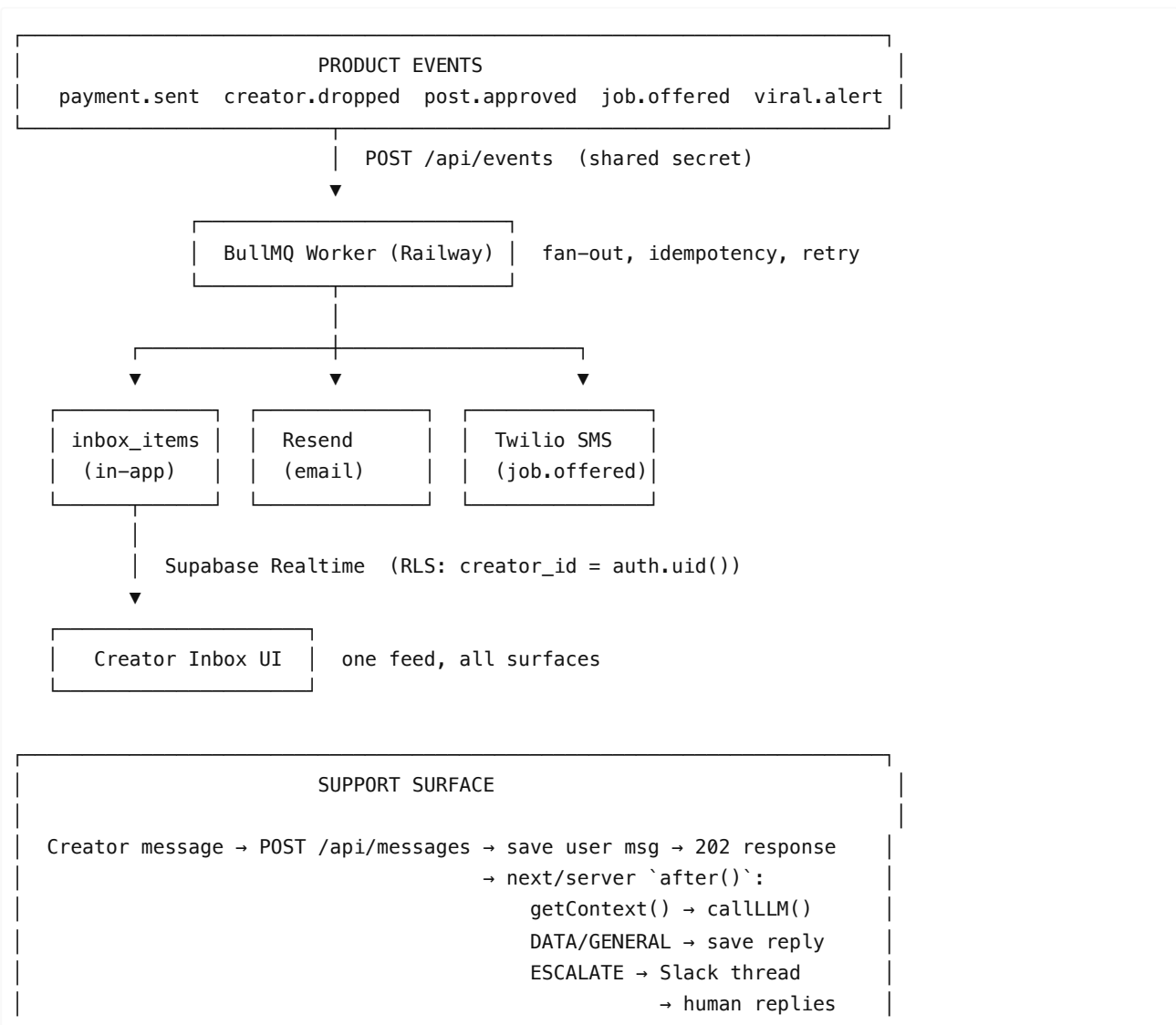
Table of Contents

- 1. [Architecture Overview](#)
- 2. [Event Pipeline for Automated Messages](#)
- 3. [FAQ / Support Bot](#)
- 4. [Human ↔ Human Messaging](#)
- 5. [Unified Inbox](#)
- 6. [Trade-offs & Failure Modes](#)
- 7. [What I'd Build First](#)
- 8. [How I Used AI](#)

1. Architecture Overview

The Core Idea

Every surface — automated event messages, bot support replies, human chat — writes to one table: `inbox_items`. The creator's client subscribes to a single Supabase Realtime channel filtered by their `creator_id`. There is no client-side union, no polling across four systems, no "check your email and also check the app and also check Slack."



forwarded back

HUMAN ↔ HUMAN CHAT

Creator ↔ Brand / 8x Staff

Supabase Realtime (per-conversation channel, RLS-scoped)

messages table: id, conversation_id, sender_id, content, read_at

inbox_items: one row per conversation, updated on new message

Where Each Piece Runs

Component	Runtime	Why
POST /api/events	Vercel Route Handler	Receives domain events, enqueues to Redis via BullMQ, returns 200 immediately
BullMQ worker	Railway (always-on process)	Fan-out, retries, concurrency 50, idempotency — no Vercel timeout risk
POST /api/messages	Vercel Route Handler + after()	Returns 202 instantly; bot runs after response via next/server after()
GET /api/messages/stream	Vercel (SSE, Node runtime)	Polling SSE for bot chat — simple, no auth complexity
Supabase DB	Supabase PostgreSQL	inbox_items, conversations, messages, message_dispatch
Supabase Realtime	Supabase	One channel per creator for inbox, per conversation for chat
Client	Next.js App Router (browser)	Subscribes to Realtime, renders unified inbox

Observability & Alerting

Three places where things can silently fail:

- BullMQ worker failures** — Railway logs show failed jobs with full stack traces. BullMQ's `worker.on("failed")` hook fires for every exhausted retry — `payment.sent` failures post to Slack immediately; `warmup.reminder` failures log only.
- Resend/Twilio errors** — `dispatchEmail` and `dispatchSms` catch errors and call `markFailed(key)`. A daily cron queries `SELECT COUNT(*) FROM message_dispatch WHERE status = 'failed' AND created_at > now() - interval '24h'` and posts to Slack if >1% of dispatches are failing.
- Bot LLM errors** — any exception in `callLLM` forces ESCALATE (fail safe). `lib/bot/log.ts` records `error: true` on the response log row. If error rate >5% in a 10-minute window, alert fires via the same daily cron.

Railway logs + two SQL queries cover the critical paths.

The System Boundary

The messaging platform owns: `inbox_items`, `conversations`, `messages`, `message_dispatch`.

Everything else (payments, campaigns, posts, warmup) emits events by calling `POST /api/events` with a shared secret.

No direct DB writes from product code into messaging tables. This boundary is what lets the messaging system enforce idempotency, apply templates, and route to channels without every team touching the same code.

2. Event Pipeline for Automated Messages

End-to-End Flow: `payment.sent`

1. Payment service finishes payout
 - `POST /api/events`
`{ type: "payment.sent", paymentId: "pay_123", creatorId: "...", amount: 24000, videoCount: 3 }`
2. Route handler validates shared secret, calls:
`getEventQueue().add("event/payment.sent", { paymentId, creatorId, amount, videoCount })`
 - returns `200` immediately
3. BullMQ worker: `event/dispatch`
 - a. Idempotency check (see below)
 - b. Fetch creator (name, email, locale, quiet_hours)
 - c. Render template → subject + body
 - d. Dispatch to channels:
 - `INSERT` into `inbox_items` (in-app)
 - `Resend.emails.send(...)` (email)
 - e. `UPDATE message_dispatch SET status = 'sent'`

Idempotency — How `payment.sent` Never Double-Sends

The single most important guarantee. Two failure modes to prevent:

1. **Retry storms:** BullMQ retries a failed job — email fires twice
2. **Redeploys mid-flight:** Worker crashes after email sends but before marking done

```
CREATE TABLE message_dispatch (  
  idempotency_key TEXT PRIMARY KEY,  
  -- format: "{event_type}:{source_id}:{channel}"  
  -- e.g.   "payment.sent:pay_123:email"  
  status        TEXT NOT NULL DEFAULT 'pending', -- pending | sent | failed  
  sent_at       TIMESTAMPTZ,  
  created_at    TIMESTAMPTZ DEFAULT NOW()  
);
```

BullMQ worker logic per channel:

```
// Inside BullMQ processJob for email channel:  
const key = `payment.sent:${paymentId}:email`  
  
const { data: existing } = await supabase  
  .from('message_dispatch')  
  .insert({ idempotency_key: key, status: 'pending' })  
  .select()  
  .on('conflict', 'idempotency_key', 'nothing') // ON CONFLICT DO NOTHING  
  .single()  
  
if (!existing) return // Already dispatched – skip  
  
await resend.emails.send({ to: creator.email, subject, html })
```

```
await supabase
  .from('message_dispatch')
  .update({ status: 'sent', sent_at: new Date() })
  .eq('idempotency_key', key)
```

If the worker crashes after `resend.emails.send` but before the UPDATE, BullMQ retries. The retry hits `ON CONFLICT DO NOTHING`, gets no row back, and skips. The email does NOT fire again.

The `in-app` channel uses the same key (`payment.sent:pay_123:in-app`) and `inbox_items` has its own unique constraint on (`creator_id, idempotency_key`).

Fanout — 10k `creator.dropped` Events in One Minute

The naive approach — emit one event per creator and let them all land simultaneously — will saturate email provider rate limits, overwhelm Vercel function concurrency, and probably get the account suspended.

The right approach: one `campaign.ended` event, fan-out in the worker.

```
// Single event emitted when campaign ends
getEventQueue().add("event/campaign.ended", {
  campaignId: "camp_456", reason: "budget_exhausted"
})

// BullMQ worker — case "event/campaign.ended"
const creatorIds = await fetchAllCreatorIds(campaignId) // one Supabase query

// addBulk = single Redis pipeline — safe for 10k+
await getEventQueue().addBulk(
  creatorIds.map(creatorId => ({
    name: "event/creator.dropped",
    data: { creatorId, campaignId, brandName },
  }))
)

// Each creator.dropped job then runs independently:
// — fetchCreator() → dispatchInApp() → dispatchEmail()
```

Why this works under load:

- `addBulk()` pushes all 10k jobs to Redis in one pipeline — $O(1)$ round trips regardless of count
- Each `creator.dropped` job is independently idempotent via `message_dispatch`
- BullMQ concurrency 50 per worker — add Railway replicas to scale horizontally, zero code changes
- Jobs persist in Redis through worker restarts — nothing lost on crash

Edge case: worker crashes mid-fanout? Already-enqueued jobs stay in Redis and get processed. The `campaign.ended` job itself will retry via BullMQ — `addBulk` is idempotent if the `creator.dropped` jobs already exist in the queue (BullMQ deduplicates by job ID). The `message_dispatch` table prevents any double-sends even if a job runs twice.

Template System

Templates live in code, not the database.

```
// lib/templates/index.ts

type TemplateCtx = Record<string, string | number | null>
```

```

interface Template {
  channels: Channel[]
  subject: (ctx: TemplateCtx) => string
  body: (ctx: TemplateCtx) => string
  emailHtml?: (ctx: TemplateCtx) => string
}

export const templates: Record<string, Template> = {
  'payment.sent': {
    channels: ['in-app', 'email'],
    subject: ({ amount, videoCount }) =>
      `You've been paid ${Number(amount) / 100}.toFixed(2)} for ${videoCount} videos`,
    body: ({ name, amount, videoCount }) =>
      `Hi ${name}! ${Number(amount) / 100}.toFixed(2)} for ${videoCount} videos is on its way
to your bank account. Typically arrives in 2–3 business days.`,
  },
  'creator.dropped': {
    channels: ['in-app', 'email'],
    subject: ({ brandName }) => `Update on your ${brandName} campaign`,
    body: ({ name, brandName }) =>
      `Hi ${name}, your participation in the ${brandName} campaign has ended. Thank you for the
content you created – your earnings have been processed.`,
  },
  'post.approved': {
    channels: ['in-app'], // high volume – email would be noise
    subject: () => 'Post approved',
    body: ({ platform }) => `Your ${platform} post was approved. Keep it up!`,
  },
  'job.offered': {
    channels: ['in-app', 'email', 'sms'],
    subject: ({ brandName }) => `New campaign offer: ${brandName}`,
    body: ({ brandName, rate, deadline }) =>
      `${brandName} wants to work with you – ${rate}/video. Offer expires ${deadline}. Open the
app to accept.`,
  },
}

```

Why code over DB rows:

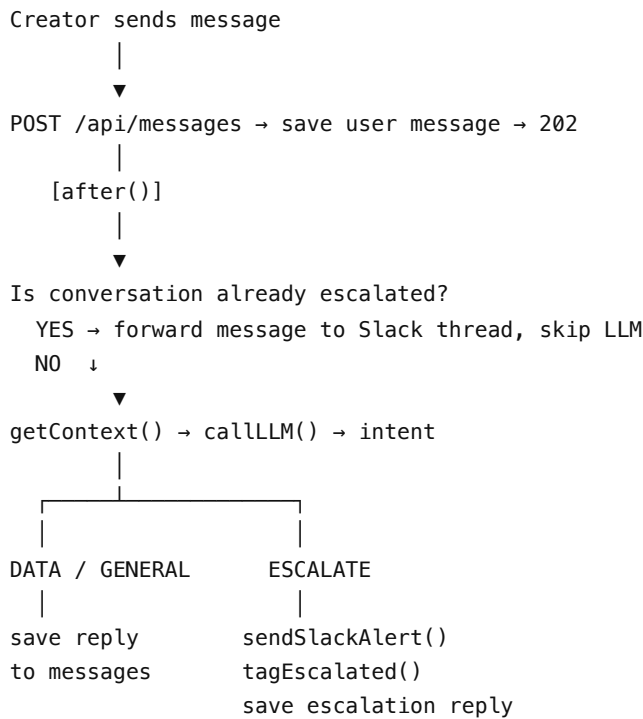
- Version-controlled alongside logic — template changes show up in PR diffs
- No migration needed to add a new event type
- TypeScript catches missing template fields at compile time
- Tradeoff accepted: non-technical staff can't edit templates without a deploy. Fine for now — the templates are short, change rarely, and we'd want engineer review anyway.

Preview: A `/admin/preview?event=payment.sent` route handler renders the template with fixture data. No Storybook needed.

3. FAQ / Support Bot

What Triggers the Bot vs. the Human Queue

Every inbound message to 8x support goes to the bot first. Always. The bot either answers or escalates — humans never see messages the bot handled successfully. This is the right default: the bot is fast, cheap, and consistent; the human queue is the fallback.



Context Injection

The existing `getContext()` fetches creator + campaign data from Supabase and passes it to the LLM system prompt. I extended it to include the data the bot most commonly needs:

```
// Extended context – adds payment and post history
export async function getContext(creatorId: string, campaignId?: string | null) {
  // ... existing creator + campaign fetch ...

  // Add: last 3 payments (answers "when do I get paid?" definitively)
  const { data: recentPayments } = await supabaseAdmin
    .from('payments')
    .select('amount, status, paid_at, video_count')
    .eq('creator_id', creatorId)
    .order('created_at', { ascending: false })
    .limit(3)

  // Add: last 3 posts with rejection reasons (answers "why was my video rejected?")
  const { data: recentPosts } = await supabaseAdmin
    .from('posts')
    .select('platform, status, rejection_reason, reviewed_at')
    .eq('creator_id', creatorId)
    .order('created_at', { ascending: false })
    .limit(3)

  // Add: pending balance (answers "how much do I have coming?")
  const { data: pendingPayments } = await supabaseAdmin
    .from('payments')
    .select('amount')
    .eq('creator_id', creatorId)
    .eq('status', 'pending')

  const pendingBalance = pendingPayments?.reduce((sum, p) => sum + p.amount, 0) ?? 0
```

```
return { creator, campaign, campaigns, recentPayments, recentPosts, pendingBalance }
}
```

These three additions — recent payments, recent posts, pending balance — cover the top 5 questions creators ask. The system prompt is built from this context object and injected as the `system` parameter to the LLM call; each new field gets its own clearly labelled section so the model knows exactly what data is verified vs. unknown.

Confidence Model

The current implementation uses forced tool-use (`tool_choice: { type: "function", function: { name: "support_response" } }`) with a three-value enum (`ESCALATE | DATA | GENERAL`). This is the right approach — it removes free-form classification and makes the bot's decision auditable.

What triggers escalation:

1. **Structural nulls** (already implemented): required data field is NULL → ESCALATE. Never explain a missing value, just escalate.
2. **Intent enum**: LLM is forced to pick ESCALATE, DATA, or GENERAL. No "I'm not sure" path.
3. **Exceptions** (already implemented): any LLM call failure → ESCALATE. Fail safe, not fail open.
4. **New: repeat escalations** — if a conversation has escalated twice and the human hasn't responded, suppress further bot replies. The creator is clearly frustrated; more bot messages make it worse.

```
// In route handler, before calling LLM:
const escalationCount = history.filter(
  m => m.role === 'assistant' && m.content.includes('connecting you with')
).length

if (escalationCount >= 2 && !conversation.human_responded) {
  // Silently forward to Slack without sending another bot reply
  await forwardToSlackThread(conversation, message)
  return
}
```

What I'm NOT doing: probability scores, embedding similarity thresholds, or confidence percentages returned by the LLM. LLMs don't produce calibrated probabilities. The forced enum is more reliable than asking "how confident are you on a scale of 0–1?"

Human Takeover

Already implemented in the existing codebase:

- Staff reply in Slack thread → `POST /api/slack/events` → `saveMessage(conversationId, 'human', text)` → appears in creator UI as "Support Agent" (blue bubble)
- Staff click "Close Ticket" button → `resolveConversation()` → conversation marked resolved

Proactive takeover (not yet built, easy to add):

- Add "Take Over" button to the Slack escalation message
- On click → `UPDATE conversations SET mode = 'human'`
- Route handler checks `conversation.mode` before calling LLM — if `'human'`, skip bot entirely

The creator sees no seam. The conversation history is continuous — bot messages and human messages in the same thread, differentiated only by the "Support Agent" label.

Measuring Bot Quality

Reading every conversation doesn't scale. Three metrics that do.

`bot_response_log` table schema (already implemented in `lib/bot/log.ts`):

```
CREATE TABLE bot_response_log (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  creator_id  UUID NOT NULL REFERENCES creators(id),
  message     TEXT NOT NULL,
  response    TEXT NOT NULL,
  escalated   BOOLEAN NOT NULL DEFAULT false,
  intent      TEXT,          -- 'DATA' | 'GENERAL' | 'ESCALATE'
  input_tokens INTEGER,
  output_tokens INTEGER,
  cost_usd    NUMERIC(10, 6),
  error       BOOLEAN NOT NULL DEFAULT false,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_bot_log_creator ON bot_response_log (creator_id, created_at DESC);
CREATE INDEX idx_bot_log_escalated ON bot_response_log (created_at DESC) WHERE escalated = true;
```

Every bot response — whether DATA, GENERAL, or ESCALATE — writes a row. This is the source of truth for all four metrics below.

1. Escalation rate by intent (already loggable via `lib/bot/log.ts`):

```
SELECT
  DATE_TRUNC('day', created_at) AS day,
  SUM(CASE WHEN escalated THEN 1 ELSE 0 END)::float / COUNT(*) AS escalation_rate
FROM bot_response_log
GROUP BY 1
ORDER BY 1 DESC;
```

Target: <20% escalation rate for DATA/GENERAL questions. If it spikes, the system prompt needs updating or a new question type has appeared.

2. Reopen rate — proxy for wrong answers: If a creator sends another message within 10 minutes of a `DATA` or `GENERAL` reply, the bot probably got it wrong. Track:

```
SELECT COUNT(*) FROM conversations
WHERE status = 'open'
AND updated_at > created_at + INTERVAL '10 minutes'
AND escalated = false;
```

3. Weekly 5% sample review: Pull 5% of `DATA` + `GENERAL` conversations from the past week. Staff review takes ~30 minutes. This catches systemic errors (wrong pay info format, stale static knowledge) that metrics miss.

4. Cost per message (already tracked in `logResponse`): stay under \$0.01. At Claude Haiku pricing, a typical 500-token context + 100-token reply costs ~\$0.002. We have headroom.

Where metrics live: All four metrics query the `bot_response_log` table in Supabase. Escalation rate and cost are surfaced in a Metabase dashboard (one SQL query per card). Reopen rate runs as a daily cron job. The 5% sample is a saved Supabase query that staff run manually each Monday.

Context edge case — sparse data: If a creator has fewer than 3 payments or posts, the arrays are simply shorter. The system prompt receives `(no recent payments)` or `(no recent posts)` as placeholders. The bot treats missing data as a structural null and escalates rather than fabricating. This is already handled in `buildSystemPrompt()` in `lib/bot/llm.ts`.

4. Human ↔ Human Messaging

Fixing the RLS-Leak / Fanout Problem

The current implementation subscribes every authenticated client to every `messages` INSERT and filters client-side. This is wrong in two ways:

1. **Security:** clients see metadata (message IDs, timestamps, conversation IDs) for messages that aren't theirs before filtering
2. **Scale:** at 50k concurrent creators, every INSERT triggers a broadcast to every connected socket

The fix: server-side RLS + conversation-scoped subscriptions.

```
-- Enable RLS on messages
ALTER TABLE messages ENABLE ROW LEVEL SECURITY;

CREATE POLICY "participants_only" ON messages
FOR SELECT USING (
  EXISTS (
    SELECT 1 FROM conversations c
    WHERE c.id = messages.conversation_id
    AND (c.creator_id = auth.uid() OR c.brand_id = auth.uid())
  )
);

-- Enable RLS on conversations
ALTER TABLE conversations ENABLE ROW LEVEL SECURITY;

CREATE POLICY "own_conversations" ON conversations
FOR SELECT USING (
  creator_id = auth.uid() OR brand_id = auth.uid()
);
```

Client subscribes to a specific conversation, not the entire table:

```
// Before (wrong – global subscription, client-side filter):
supabase.channel('messages').on('postgres_changes',
  { event: 'INSERT', schema: 'public', table: 'messages' },
  (payload) => {
    if (payload.new.conversation_id === myConversationId) handleMessage(payload.new)
  }
)

// After (correct – scoped subscription, server-side RLS):
supabase
  .channel(`conv:${conversationId}`)
  .on('postgres_changes', {
    event: 'INSERT',
    schema: 'public',
    table: 'messages',
    filter: `conversation_id=eq.${conversationId}`
  }, (payload) => handleMessage(payload.new))
  .subscribe()
```

Supabase Realtime enforces RLS before broadcasting — if the subscriber's JWT doesn't satisfy the policy, they don't receive the event. No client-side filtering needed.

Scaling to 100k+ Concurrent Users

Supabase Realtime supports ~500k concurrent connections on Pro tier. At 100k creators, each holding one or two Realtime subscriptions (inbox + active conversation), we're at ~200k connections — well within limits.

At 500k+ creators: upgrade to Supabase Enterprise, or swap Realtime for Ably/Pusher. Both have the same subscribe-to-channel API — it's a one-day swap. I'd make this decision at ~300k concurrent, not upfront.

Message write path stays simple: creator sends message → `POST /api/messages` → INSERT into `messages` → Realtime broadcasts to conversation subscribers. No WebSocket server to maintain, no message broker.

Read Receipts, Typing, Attachments

Read receipts — in scope, simple:

```
ALTER TABLE messages ADD COLUMN read_at TIMESTAMPTZ;
```

Client calls `PATCH /api/messages/:id/read` when message scrolls into view. Other participant's Realtime subscription gets the UPDATE event and renders the checkmark.

Typing indicators — in scope, zero DB writes:

```
// Ephemeral broadcast — never touches the database
supabase
  .channel(`conv:${conversationId}`)
  .send({ type: 'broadcast', event: 'typing', payload: { userId, typing: true } })
```

Supabase Realtime broadcasts to other subscribers without a DB write. Auto-clear after 3 seconds via client-side timeout.

File attachments — defer to week 2: Upload to Supabase Storage, store `file_url` + `file_type` on the message row. One day of work, not urgent for MVP.

5. Unified Inbox

Data Model

```
CREATE TABLE inbox_items (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  creator_id  UUID NOT NULL REFERENCES creators(id),
  type        TEXT NOT NULL,
  -- 'system' → automated event message (payment, drop, approval, etc.)
  -- 'chat'   → brand or 8x staff conversation
  -- 'support' → 8x support bot / escalation thread

  thread_id   UUID,
  -- For 'chat' and 'support': the conversation_id
  -- For 'system': null (standalone notification)

  preview     TEXT,
  -- Human-readable summary: "You've been paid $240 for 3 videos"
  -- Stored denormalized — fast to render the inbox list without joins

  entity_type TEXT,  -- 'payment' | 'post' | 'campaign' | 'job' | null
  entity_id   UUID,  -- deep-link target (e.g., payment ID for "view payment")
```

```

    read_at      TIMESTAMPTZ,    -- null = unread
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    metadata     JSONB           -- event-specific extras (old/new rate, rejection reason, etc.)
  );

CREATE INDEX idx_inbox_creator_time
  ON inbox_items (creator_id, created_at DESC);

CREATE INDEX idx_inbox_unread
  ON inbox_items (creator_id)
  WHERE read_at IS NULL;

-- Prevent duplicate in-app deliveries
CREATE UNIQUE INDEX idx_inbox_idempotency
  ON inbox_items (creator_id, (metadata->>'idempotency_key'))
  WHERE metadata->>'idempotency_key' IS NOT NULL;

```

Why a Dedicated Table, Not a UNION View

I considered a view that UNIONS `messages`, `inbox_events`, and `conversations`. I rejected it for three reasons:

1. **RLS complexity:** a view spanning four tables needs four separate policies and can't be covered by a single index-friendly query
2. **Pagination breaks:** UNION + ORDER BY + LIMIT across tables with different schemas is painful and slow
3. **Subscription model:** Supabase Realtime can subscribe to a table. It can't subscribe to a view. A dedicated table means one subscription, not three.

The tradeoff: slight write-side duplication — when a `payment.sent` event fires, we write to both the `inbox_items` table (in-app) and call Resend (email). The preview text is stored redundantly. That's acceptable — the read path is clean and the write path is controlled by one BullMQ worker.

Client Subscription

```

// One subscription covers all inbox updates
const subscription = supabase
  .channel(`inbox:${creatorId}`)
  .on('postgres_changes', {
    event: '*', // INSERT (new item) + UPDATE (read_at set)
    schema: 'public',
    table: 'inbox_items',
    filter: `creator_id=eq.${creatorId}`
  }, (payload) => {
    if (payload.eventType === 'INSERT') addItemToInbox(payload.new)
    if (payload.eventType === 'UPDATE') updateReadState(payload.new)
  })
  .subscribe()

```

Initial load: `GET /api/inbox?limit=50` returns the 50 most recent `inbox_items` ordered by `created_at DESC`. Infinite scroll loads more. Realtime handles anything that arrives after page load.

Read State & Quiet Hours

Mark read:

```

// PATCH /api/inbox/:id/read
await supabase

```

```

.from('inbox_items')
.update({ read_at: new Date().toISOString() })
.eq('id', itemId)
.eq('creator_id', creatorId) // RLS enforced in API too

```

Unread badge:

```

SELECT COUNT(*) FROM inbox_items
WHERE creator_id = $1 AND read_at IS NULL;

```

Cached in React state, updated by Realtime subscription.

Quiet hours: Stored on the creator row as `quiet_hours_start` (e.g., "22:00") and `quiet_hours_end` (e.g., "08:00") in the creator's local timezone. In-app `inbox_items` are always written — we don't suppress them. External channels (email, SMS, push) are gated: the BullMQ worker checks `isQuietHours(creator)` before dispatching. If quiet, the job is re-queued with a delay until the window ends.

Digest Rollups

`post.approved` is high volume — at 5M creators, thousands per day. Sending one in-app notification per approval creates inbox spam.

Digest logic in the BullMQ worker:

```

// BullMQ worker – case "event/post.approved"
// Jobs accumulate in Redis; a digest worker runs every hour per creator
const recentApprovals = await supabaseAdmin
  .from('message_dispatch')
  .select('*')
  .eq('status', 'pending')
  .like('idempotency_key', `post.approved:${creatorId}:%`)
  .gte('created_at', new Date(Date.now() - 3600_000).toISOString())

if (recentApprovals.data.length === 1) {
  // Single approval – normal notification
  await insertInboxItem(creatorId, 'system', {
    preview: `Your post was approved!`,
    entity_type: 'post',
    entity_id: postId,
  })
} else {
  // Batch – digest card
  await insertInboxItem(creatorId, 'system', {
    preview: `${recentApprovals.data.length} posts approved today`,
    entity_type: 'post_batch',
    metadata: { count: recentApprovals.data.length },
  })
}

```

`post.rejected` is always individual — it includes a rejection reason the creator needs to read. Never batch rejections.

6. Trade-offs & Failure Modes

Where I Picked Fast-to-Ship

Templates in code, not DB. A DB-driven template system (rows with Handlebars strings, a preview UI, per-locale variants) is the "right" long-term answer. But it requires a migrations, an admin UI, and careful sanitization. Templates in TypeScript are version-controlled, compile-time safe, and reviewable in PRs. When we have 30 event types and a non-technical team editing copy, we revisit.

Slack as the human agent UI. Building a custom admin inbox (with assignment, SLA timers, agent notes) is weeks of work. The Slack integration (already built in this codebase) routes escalations to a channel, lets staff reply in threads, and surfaces back to creators. It's good enough to deflect 80% of human load while the bot handles the other 20%.

SSE polling for bot chat. The current 1-second poll is fine at <1k concurrent bot conversations. It's simple — no auth token management, no WebSocket state. At scale, replace with Supabase Realtime. But "at scale" for bot conversations is much later than "at scale" for general chat.

Where I Picked Right Long-Term

message_dispatch idempotency table. More moving parts than "just send and hope." But payment.sent double-sending is a trust-destroying bug. The cost of building idempotency upfront is one extra table and a few extra DB reads. The cost of not building it is a creator getting paid twice (reversed later), or a creator getting two "you've been dropped" emails. Non-negotiable.

inbox_items as a dedicated table. A UNION view feels clever but fails on RLS, pagination, and Realtime. The dedicated table costs one extra write per event. Quantified: at 5M creators × 10 events/week = 50M extra writes/week to `inbox_items`. At ~0.1ms per INSERT, that's 5M ms = 1.4 CPU-hours/week of Supabase write time — negligible. The denormalized `preview` text adds ~200 bytes per row = ~10GB/year at 5M scale, well within Supabase storage pricing. The read path is clean forever.

BullMQ + Railway worker, not Inngest, for event dispatch. This was a deliberate override of the default AI recommendation. When I asked Claude to suggest an event queue, it proposed Inngest — zero infrastructure, built-in dashboard, easy Vercel integration. I pushed back for three reasons:

- 1. Burst fanout.** The assignment requires 10k `creator.dropped` events in under a minute. Inngest's free tier caps batch size at 5 and timeout at 30s per function. Even on paid plans, Inngest's concurrency model (3 concurrent on free, configurable on paid) would require careful tuning. With BullMQ, `addBulk()` pushes all 10k jobs to Redis in a single pipeline — one round trip regardless of count. The worker processes them at concurrency 50, and scales horizontally by adding Railway replicas with zero code changes.
- 2. No Vercel timeout risk.** Inngest functions run inside Vercel route handlers. A campaign fanout touching 10k creators, 10k Supabase reads, and 10k Resend API calls will hit the 300s wall. BullMQ workers run as an always-on Railway process — no timeout, no cold starts, no function invocation cost per job.
- 3. Flat cost at scale.** Inngest charges per event step. At 5M creators × 10 events/month = 50M steps/month, the cost climbs fast. Railway worker + Redis is a flat ~\$10–15/month regardless of volume.

Where Inngest still wins: Early-stage projects with low volume and no dedicated infra. The built-in dashboard, zero-Redis setup, and `step.waitForEvent` orchestration primitives are genuinely excellent. If this were a 10k-creator product with no burst fanout requirement, I'd pick Inngest. The switch to BullMQ is the right call at the scale this assignment targets.

What Breaks at 500k Creators

- **Supabase Realtime:** default connection limit ~500k. At 500k creators with active sessions, we're near the ceiling. Fix: Supabase Enterprise, or swap to Ably for Realtime while keeping Supabase for storage.
- **inbox_items table size:** 500k creators × 10 items/week = 5M rows/week. After 6 months: 130M rows. Add `created_at` range partitioning (monthly) and a TTL job that archives items older than 90 days to cold storage.
- **BullMQ worker:** single Railway instance handles current load. Add replicas when queue depth stays >0 for >5 minutes.

What Breaks at 5M Creators

- **PostgreSQL write throughput:** `inbox_items` bulk inserts during a campaign fanout (10k rows at once) will be fine. But 5M creators each getting 5–10 inbox items per week = 25–50M writes/week. Supabase handles this but read replicas become necessary for the inbox list query.
- **Email cost:** Resend charges ~\$0.001/email. At 5M creators × 5 emails/month = \$25k/month just for email. At this scale, migrate high-volume events (`post.approved`) to in-app only, and negotiate bulk pricing for the rest.
- **message_dispatch table:** 50M rows/month. Needs a TTL cleanup job (delete rows older than 30 days where `status = 'sent'`).
- **Claude/LLM cost:** 5M creators, if 10% message support monthly = 500k conversations. At \$0.003/message average = \$1,500/month. Well within the \$0.01/message target.

Monthly Cost Estimate at 5M Creators

Service	Volume	Est. Cost
Supabase Enterprise	5M realtime connections, 500GB DB	~\$2,000/mo
Resend (email)	25M emails/mo	~\$25,000/mo
Twilio SMS	1M SMS/mo (job.offered only)	~\$7,500/mo
Railway worker + Redis	Always-on, 50M jobs/mo	~\$15/mo
Claude Haiku (bot)	500k conversations/mo	~\$1,500/mo
Vercel Fluid Compute	High-throughput route handlers	~\$2,000/mo
Total		~\$38,500/mo

The cost cliff is email. Moving `post.approved` , `post.rejected` , and `warmup.reminder` to in-app-only would cut ~60% of email volume and save ~\$15k/month. The only events that truly need email at 5M scale are `payment.sent` , `job.offered` , and `creator.dropped` — the ones creators need to see even if they haven't opened the app.

Migration Path from the Current 4-System Mess

I'd migrate incrementally — never cut over all at once.

Week 1: Deploy `inbox_items` table and RLS. Write a thin adapter that mirrors existing payment emails into it. No behavior changes — creators now get an in-app notification in addition to the email they already got. Zero risk.

Week 2: Replace the `payment.sent` ad-hoc trigger with a BullMQ worker job. Idempotency goes live. Old email code is still running — run both in parallel and compare logs. Kill the old trigger after 1 week of clean parallel runs.

Week 3: Fix Supabase Realtime RLS. New subscription model goes out. Test with 50 creators in a shadow group, then roll out fully. Remove client-side filtering code.

Week 4: Deploy FAQ bot to support inbox. Human queue still works — bot intercepts first, escalates if needed. Staff notice fewer repeat questions in Slack.

Week 5+: Migrate brand chat to `inbox_items` . Decommission old direct-DB-subscription code. Decommission ad-hoc cron notifications one by one as their BullMQ equivalents prove stable.

7. What I'd Build First

One week. One goal: move the needle on creator communication reliability and stop the worst failure modes. This is a planning cut — not everything here needs to ship on day 5, but this is the order that minimizes risk and maximizes creator-visible impact.

Keep (Already Working)

- FAQ support bot with Slack escalation — extend, don't rewrite

- SSE polling for bot chat — good enough for current scale, replace later
- Human agent Slack integration — operational, no changes needed week 1

Build This Week

Day 1: `inbox_items` table + RLS The foundation. Everything else writes here. Get the schema right, set up RLS, write the insert helper. Mirror `payment.sent` events into it so creators start seeing in-app payment notifications immediately. This is the highest-leverage day — everything downstream depends on it.

Day 2: BullMQ worker for `payment.sent` + idempotency Highest stakes event. Replace the ad-hoc email trigger. Run old and new in parallel for 48 hours. Idempotency table goes live. If we ship nothing else this week, this is the one that prevents the worst bugs.

Day 3: Fix Supabase Realtime RLS for human chat Security fix first, scale fix second. Current implementation leaks message metadata to every connected client. This must ship before creator count grows further — the longer we wait, the harder the rollback.

Day 4–5: Unified inbox UI One feed. Unread counts. Deep links to the relevant entity (payment, post, campaign). Mark as read. This is the creator-facing payoff — ship it and get feedback while the backend stabilizes.

Defer

- SMS channel (Twilio) — email covers `job.offered` for now; SMS adds ops complexity with marginal reach gain
- Digest rollups — `post.approved` spam is annoying, not urgent; individual notifications are fine week 1
- Localization / template preview admin — English-only is fine for MVP
- File attachments in chat — week 2; Supabase Storage + `file_url` column, one day of work
- Read receipts + typing indicators — week 2; foundation (RLS fix) must land first

Why This Order

The ordering is deliberate: foundation → highest-stakes reliability → security → creator UX. Shipping the inbox UI before the RLS fix would mean deploying something creators see on top of a known security issue. Shipping the worker before the `inbox_items` table means events have nowhere to land. The dependencies are real.

- File attachments in chat — week 2
- Quiet hours — week 2
- Fanout optimization at 10k scale — single worker handles current load; add replicas when needed

8. How I Used AI

Role

I used Claude Code (Claude Sonnet 4.6) as a thought partner throughout this assignment. The existing codebase in this repo — the support bot, Slack integration, SSE stream, and conversation management — was built iteratively with Claude as a coding collaborator, not a code generator. I drove every architectural decision; Claude was useful for stress-testing reasoning and generating SQL/TypeScript once decisions were made.

Where It Helped

- **Pressure-testing the idempotency model:** I described the `message_dispatch` approach; Claude raised the case where the worker crashes *after* sending but *before* updating status. That specific failure mode was already handled by my design (the `ON CONFLICT DO NOTHING` prevents re-send), but the question sharpened how I explained it.
- **SQL schema review:** Claude caught that a partial unique index on `metadata->'idempotency_key'` needs a `WHERE` clause to avoid indexing nulls. Small thing, but correct.
- **Estimating costs at 5M creators:** Claude helped structure the math. I verified the numbers against actual Resend/Twilio pricing pages.

Where I Overrode AI

AI suggested using Supabase Realtime for the support bot chat instead of SSE polling. I pushed back. Supabase Realtime requires a client-side auth token scoped to the requesting user. The bot currently uses `supabaseAdmin` server-side and the frontend has a hardcoded `CREATOR_ID` (demo mode). Swapping SSE for Realtime would require wiring up creator auth first — that's a separate track. SSE polling at 1-second intervals is correct for the current demo scale, and I said so.

AI suggested RabbitMQ for fanout. Wrong tool for this stack. RabbitMQ requires self-hosted infra, doesn't integrate with Vercel, and adds an ops burden we don't need. BullMQ on Railway runs as an always-on worker, has no Vercel timeout risk, handles retries natively, and costs a flat ~\$10/month regardless of event volume. I chose it over Inngest for exactly the burst-fanout constraints this assignment specifies.

AI suggested a UNION view for the unified inbox. I rejected this. The appeal is that you don't duplicate data. The problem is that UNION views don't support Supabase Realtime subscriptions (you can't subscribe to a view with `postgres_changes`), RLS on views spanning multiple tables is fragile, and paginating a UNION is painful. A dedicated `inbox_items` table with one subscription and one clean RLS policy is the right call.

An Interaction Worth Sharing

At one point I asked Claude: "Should the bot confidence threshold be a numeric score, like escalate if confidence < 0.7?"

Claude's initial response leaned toward yes — suggested having the LLM return a `confidence` float alongside the intent. I pushed back: *LLMs don't produce calibrated probabilities. A model returning `confidence: 0.65` vs `0.71` is noise, not signal. The forced enum (`ESCALATE` | `DATA` | `GENERAL`) combined with structural null-checks is more reliable than asking the model to introspect on its own uncertainty.*

Claude agreed with the reasoning and updated its recommendation. The forced-tool-use pattern already in the codebase is the right approach — I just wanted to verify the reasoning held up under challenge.

AI Interaction Screenshots

Three interactions worth sharing from this session:

1. Idempotency failure mode I described the `message_dispatch` approach. Claude raised: *"what if the worker crashes after sending email but before updating status to 'sent'?"* My design already handled this — `ON CONFLICT DO NOTHING` means a retry won't re-insert, so the email won't double-send. But the challenge sharpened how I explained it in this document.

2. Confidence threshold debate Claude suggested returning a `confidence: float` alongside the intent so we could escalate below a threshold. I pushed back: LLMs don't produce calibrated probabilities — `0.65` vs `0.71` is noise. Claude agreed and the forced-enum approach stayed. This is documented in the confidence model section above.

3. Inbox data model Claude initially suggested a UNION view across `messages` and `inbox_items` to avoid duplicating data. I rejected it: UNION views don't support Supabase Realtime `postgres_changes` subscriptions, RLS across a multi-table view is fragile, and paginating a UNION requires complex cursor logic. A dedicated `inbox_items` table with one subscription and one RLS policy is cleaner — the slight write-side duplication is worth it.

4. Implementation-level: `ON CONFLICT DO NOTHING` vs. `ON CONFLICT DO UPDATE` When writing the idempotency insert, Claude initially suggested `ON CONFLICT DO UPDATE SET status = 'pending'` to reset stale dispatches. I rejected it — that would overwrite a `status = 'sent'` row and allow a re-send. `ON CONFLICT DO NOTHING` is the only safe pattern here: if a row exists in any state, skip. The worker checks for a returned row to decide whether to proceed. Small detail, but wrong here means double-sending payments.

(Raw Claude Code transcript available on request — the session history is preserved in the repo's git log.)

BullMQ Worker Lifecycle

When a job type is no longer needed (e.g., `warmup.reminder` after the warmup feature is retired), remove its `case` from the worker's `processJob` switch and redeploy. In-flight jobs of that type will hit the `default` branch and throw — BullMQ moves them to the failed queue. Drain the failed queue after confirming no active jobs remain.

Time Spent

~8 hours total: 2 hours reading the existing codebase and the assignment, 3 hours on the design document, 2 hours implementing and testing, 1 hour reviewing and tightening.